WEEK 3 · LESSON 16 OF 24 · HANDOUT

Better Retrieval: Chunking, Hybrid Search and Reranking

Study time: **about 55 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=9wk7mGB_Gp4 — open the lesson online to watch inside the course.

🎓 **Go deeper:** EuroPython 2024: Fundamentals of Retrieval Augmented Generation — https://www.youtube.com/watch?v=vSwK5sBpD2c

🎓 **Go deeper:** EuroPython 2024: Deconstructing the text embedding models — https://www.youtube.com/watch?v=Lha_QCkXxm0

Your goal in this lesson: Measure retrieval quality with recall@k on labelled questions, and improve it with keyword search (BM25), hybrid search, query rewriting and reranking.

Measure retrieval on its own

Before judging answers, check whether the **right section** is among the retrieved ones. For each test question we know which section answers it:

```
HANDBOOK = {
    "Opening hours": "The Kigali Tech Hub is open Monday to Friday from 8:00 to 20:00 and on
Saturday "
                     "from 9:00 to 14:00. The hub is closed on Sundays and on public
holidays.",
    "Membership plans": "There are three plans. The Student plan costs 10,000 RWF per month and
needs a "
                        "valid student card. The Standard plan costs 25,000 RWF per month. The
Team plan "
                        "costs 90,000 RWF per month for up to five people.",
    "Joining": "To become a member, fill in the online form, bring your national ID or passport
to the "
               "front desk, and pay the first month. Your access card is ready the same day.",
    "Payment": "Fees are paid by mobile money or bank transfer before the 5th of each month.
Members who "
               "pay late for two months lose access until the balance is cleared.",
    "Cancelling": "You may cancel your membership at any time with 14 days' written notice by
email. "
                  "Fees already paid for the current month are not refunded.",
    "Wi-Fi": "The Wi-Fi network is called KTH-Members. The password is printed on your access
card and "
             "changes every three months. Guests use the separate KTH-Guest network.",
    "Meeting rooms": "Members can book the two meeting rooms through the hub app. Standard and
Team members "
                     "get 4 free hours per month; Student members get 1 free hour. Extra hours
cost 5,000 RWF.",
    "Guests": "Each member may bring up to two guests per day. Guests must sign in at the front
desk "
              "and leave by 18:00.",
    "Printing": "Black-and-white printing costs 50 RWF per page and colour printing 200 RWF per
page. "
                "Students get 100 free black-and-white pages per month.",
    "Events": "The hub runs a free coding workshop every Wednesday at 17:00 and a startup pitch
night "
              "on the last Friday of each month. Members can register in the hub app.",
    "Lockers": "Lockers can be rented for 3,000 RWF per month. Lost locker keys cost 5,000 RWF
to replace.",
    "Code of conduct": "Members must respect others, keep noise low in the quiet zone, and
never share their "
                       "access card. Harassment leads to immediate cancellation of
membership.",
    "Lost and found": "Items left behind are kept at the front desk for 30 days, then
donated.",
    "Accessibility": "The ground floor, meeting rooms and toilets are wheelchair accessible. A
lift serves "
                     "all floors. Contact the front desk to arrange other support.",
}
chunks = [{"id": i, "title": t, "text": f"{t}. {body}"} for i, (t, body) in
enumerate(HANDBOOK.items())]
print(len(chunks), "sections")
```

```
test_questions = [
    ("How much is the student membership?", "Membership plans"),
    ("Can I come to the hub on Sunday?", "Opening hours"),
    ("What is the Wi-Fi password?", "Wi-Fi"),
    ("How do I book a meeting room?", "Meeting rooms"),
    ("If I quit, do I get my money back for this month?", "Cancelling"),
    ("How many friends can I bring with me?", "Guests"),
    ("What does colour printing cost?", "Printing"),
    ("When is the coding workshop?", "Events"),
    ("What documents do I need to sign up?", "Joining"),
    ("What happens if I don't pay my fees on time?", "Payment"),
    ("I left my bag at the hub last week. Is it still there?", "Lost and found"),
    ("Is there a lift for wheelchair users?", "Accessibility"),
]
print(len(test_questions), "labelled questions")
```

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import Normalizer

texts = [c["text"] for c in chunks]
embedder = make_pipeline(TfidfVectorizer(stop_words="english", sublinear_tf=True),
                         TruncatedSVD(n_components=12, random_state=0),
                         Normalizer())                      # unit length: dot product = cosine
similarity
doc_vectors = embedder.fit_transform(texts)

def embed(text_list):
    return embedder.transform(text_list)

print("each section is now a vector of", doc_vectors.shape[1], "numbers")
```

```
def recall_at_k(search_fn, k=3):
    hits = 0
    for question, expected in test_questions:
        titles = [chunks[i]["title"] for i in search_fn(question)[:k]]
        hits += expected in titles
    return hits / len(test_questions)

def embedding_ranking(question):
    return list(np.argsort(doc_vectors @ embed([question])[0])[::-1])

for k in [1, 3, 5]:
    print(f"embeddings recall@{k}: {recall_at_k(embedding_ranking, k):.2f}")
```

Recall@k = the share of questions whose correct section appears in the top k. It is the single most useful number for a RAG system: if it is low, no prompt can save the answers.

BM25: keyword search done well

BM25 is the classic search-engine formula: it rewards rare words that appear in a document, with diminishing returns for repeated words and a correction for long documents.

```
import math
import re
from collections import Counter

STOP = {"the", "a", "an", "is", "are", "i", "my", "do", "to", "of", "and", "for", "in", "on",
"at",
        "what", "how", "can", "me", "it", "if", "with", "does", "there", "this", "be", "by",
"or"}

def tokenize(s):
    return [w for w in re.findall(r"[a-z0-9]+", s.lower()) if w not in STOP]

class BM25:
    def __init__(self, docs, k1=1.5, b=0.75):
        self.docs = [tokenize(d) for d in docs]
        self.avg = sum(len(d) for d in self.docs) / len(self.docs)
        self.df = Counter(w for d in self.docs for w in set(d))
        self.N, self.k1, self.b = len(self.docs), k1, b

    def scores(self, query):
        out = []
        for d in self.docs:
            tf, s = Counter(d), 0.0
            for w in tokenize(query):
                if w in tf:
                    idf = math.log(1 + (self.N - self.df[w] + 0.5) / (self.df[w] + 0.5))
                    s += idf * tf[w] * (self.k1 + 1) / (tf[w] + self.k1 * (1 - self.b + self.b
* len(d) / self.avg))
            out.append(s)
        return np.array(out)

bm25 = BM25([c["text"] for c in chunks])
def bm25_ranking(question):
    return list(np.argsort(bm25.scores(question))[::-1])

for k in [1, 3]:
    print(f"BM25 recall@{k}: {recall_at_k(bm25_ranking, k):.2f}")
```

On our handbook BM25 finds the right section first for about **58%** of questions (embeddings: about **67%**). Questions like "If I quit…" share no words with their section, so BM25 cannot find them. Keyword search is still strong for exact terms — names, codes, numbers — where embeddings can be vague.

Hybrid search: combine both

Reciprocal rank fusion (RRF) merges rankings without worrying about score scales: each document gets 1 ÷ (60 + its rank) from each method, and the sums are compared.

```
def hybrid_ranking(question, c=60):
    fused = Counter()
    for ranking in (embedding_ranking(question), bm25_ranking(question)):
        for rank, doc in enumerate(ranking):
            fused[doc] += 1 / (c + rank + 1)
    return [doc for doc, _ in fused.most_common()]

for name, fn in [("embeddings", embedding_ranking), ("BM25", bm25_ranking), ("hybrid",
hybrid_ranking)]:
    print(f"{name:10s} recall@1 {recall_at_k(fn, 1):.2f}   recall@3 {recall_at_k(fn, 3):.2f}")
```

Look closely at the result: on this tiny collection, hybrid search is **not** better than embeddings alone (recall@3 about 0.83 against 0.92) — BM25's mistakes pull some right answers down. On large real collections with a strong embedding model, hybrid search is usually a strong default, because keywords catch exact terms and embeddings catch paraphrases. The lesson: **never assume an improvement — measure it on your own questions.**

Chunking: what is one "document"?

Real documents are long. You split them into **chunks** before embedding:

- **Too large** — one vector must represent many topics; retrieval gets vague, prompts get long and costly.
- **Too small** — chunks lose context ("It costs 5,000 RWF" — what does?).
- Good defaults: a few hundred tokens per chunk, split at headings or paragraphs, with a little **overlap** between neighbours, and the section title copied into each chunk (as ours do).

```
def chunk_text(text, size=40, overlap=10):
    words = text.split()
    step = size - overlap
    return [" ".join(words[i:i + size]) for i in range(0, max(len(words) - overlap, 1), step)]

long_doc = " ".join(c["text"] for c in chunks[:4])
pieces = chunk_text(long_doc)
print(len(long_doc.split()), "words ->", len(pieces), "chunks")
print(pieces[1][:120], "...")
```

Query rewriting and reranking

- **Query rewriting:** ask an LLM to turn a vague or chatty question into a clean search query ("I left my bag… is it still there?" → "lost and found items kept how long"). Also useful to translate Kinyarwanda questions before searching English documents.
- **Reranking:** retrieve generously (say 20 chunks), then let a stronger model — a cross-encoder or an LLM — score each chunk against the question and keep the best 3. More accurate, a little slower.

```
import importlib.util, os, subprocess, sys

if importlib.util.find_spec("anthropic") is None:          # install once per Colab session
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", "anthropic"], check=True)

try:
    from google.colab import userdata                     # Colab: key saved in the Secrets
panel (key icon)
    os.environ["ANTHROPIC_API_KEY"] = userdata.get("ANTHROPIC_API_KEY")
except ImportError:
    pass                                                    # not in Colab: set
ANTHROPIC_API_KEY in your environment
except Exception:
    print("Add a secret called ANTHROPIC_API_KEY in Colab's Secrets panel, then run this cell
again.")

import anthropic

client = anthropic.Anthropic()        # reads ANTHROPIC_API_KEY from the environment
MODEL = "claude-haiku-5-5"            # fast and low-cost; check docs.claude.com for current
model names

def ask(prompt, system=None, max_tokens=500):
    extra = {"system": system} if system else {}
    response = client.messages.create(model=MODEL, max_tokens=max_tokens,
                                      messages=[{"role": "user", "content": prompt}], **extra)
    return response.content[0].text
```

```
def rewrite_query(question):
    return ask("Rewrite this question as a short search query of key words. Reply with the
query only.\n\n"
               f"<question>{question}</question>", max_tokens=40).strip()

print(rewrite_query("I left my bag at the hub last week. Is it still there?"))
```

Key terms

- **Recall@k** — share of questions whose correct chunk is in the top k results.
- **BM25** — a strong keyword ranking formula.
- **Hybrid search** — combining keyword and embedding rankings.
- **Reciprocal rank fusion** — merging rankings by 1 ÷ (c + rank).
- **Chunking / overlap** — splitting documents into searchable pieces that share a little text.
- **Reranking** — re-scoring retrieved chunks with a stronger model.

Remember this: Measure retrieval with recall@k first; then improve it with sensible chunking, hybrid keyword + embedding search, query rewriting and reranking.

Name: Date:

Activity

Task 1

Find the questions each method gets wrong at k = 1. Why do they fail?

Task 2

Write five new test questions that use different words from the handbook. Which method wins now?

Task 3

Try `chunk_text` with sizes 20, 40 and 80 words. How many chunks does each make?

Task 4

Use `rewrite_query` before `bm25_ranking` and measure recall@1 again.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What does recall@3 measure?

   A. Speed
   B. Answer quality
   C. The share of questions whose correct chunk is in the top 3 results
   D. Cost

2. When is keyword search (BM25) especially strong?

   A. Never
   B. For images
   C. For exact terms such as names, codes and numbers
   D. For paraphrases

3. What does hybrid search combine?

   A. Two LLMs
   B. Training and testing
   C. Keyword and embedding rankings
   D. Text and images

4. What is a problem with very large chunks?

   A. They cannot be embedded
   B. They have no words
   C. One vector must represent many topics, so retrieval gets vague
   D. They are always too cheap

5. What is reranking?

   A. Sorting alphabetically
   B. Changing the question
   C. Re-scoring many retrieved chunks with a stronger model and keeping the best
   D. Deleting chunks

6. Why rewrite a user's question before searching?

   A. To turn chatty or translated questions into clean search queries
   B. To make it longer
   C. To hide it
   D. It is required

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.